PRD AMENDMENT A

Voice Interaction System

Full Audio-First Student Interaction — Backend Requirements

Amends Document	VisioLearn Backend PRD v1.0
Amendment ID	Amendment A
New Version	v1.1
Date	March 2026
Sections Added	21 — Voice Interaction System (full new chapter)
Sections Updated	4 (Architecture), 5 (Data Models), 6 (API), 7 (AI Pipeline), 9 (Performance), 11 (Background Jobs), 14 (Testing), 17 (Acceptance Criteria), 18 (Dev Phases)
Reason	Primary users are visually impaired — full audio-first interaction must be supported; student voice input must drive all learning interactions

> **CRITICAL DESIGN PRINCIPLE:** Visually impaired students cannot see buttons, menus, or text on screen. Every single student interaction with the learning system — navigation, answering questions, requesting help, controlling playback, asking the AI — MUST be achievable through voice alone. The backend must be architected to receive, interpret, and respond to voice input as a first-class interaction channel, not as an afterthought.

21. Voice Interaction System — Complete Specification

This chapter defines the complete backend architecture for audio-first student interaction. It covers: voice data reception, speech-to-text transcription, natural language understanding (intent classification + entity extraction), dialogue state management, AI response generation (constrained to lesson content), and audio response delivery. It also defines the fallback pipeline ensuring the system remains functional when voice quality is poor.

21.1 Architecture Overview — Audio-First Interaction Loop

The interaction loop runs as follows. Every step either happens on-device (Android client) or backend-side. The backend's responsibilities are bolded:

Step	Where	What Happens	Backend Involvement
1. Wake / Trigger	On-device	Student taps screen or says wake phrase. Android activates microphone.	None
2. Voice Capture	On-device	Android records audio (PCM 16kHz mono, 16-bit). Max 15 seconds per utterance.	None
3. Transcription	On-device (primary) / Backend (fallback)	Android SpeechRecognizer (offline) transcribes speech to text locally. If offline ASR confidence < 0.65, raw audio is sent to backend transcription endpoint.	POST /voice/transcribe — Whisper small.en model transcribes audio; returns text + confidence
4. Intent Classification	Backend	Transcribed text is sent to backend NLU endpoint. Backend classifies intent and extracts entities (answer choice, unit reference, etc.).	POST /voice/interpret — NLU pipeline; returns intent + entities + confidence
5. Action Dispatch	Backend	Based on intent, backend performs the required action: fetch answer feedback, retrieve next unit, answer a free-form question from lesson content, etc.	Dispatches to relevant service; returns structured action_response
6. TTS Response	On-device	Android client receives response text from backend. Android TTS engine reads it aloud to student.	Backend returns response_text; client handles TTS playback
7. State Update	On-device + Backend	Interaction is logged; session state updated on both device and backend.	POST /voice/session/event stores interaction log

> **OFFLINE FIRST — VOICE:** Voice interaction MUST degrade gracefully offline. Steps 1–4 (capture + transcription + basic intent matching) MUST work fully offline using on-device Android SpeechRecognizer and a downloaded intent model. Backend voice endpoints are only called when connectivity is available. The system must NOT become non-functional when offline — see Section 21.9 for full offline fallback specification.

21.2 Voice Session Concept & State Machine

A Voice Session is the stateful context of a student's current learning interaction. It tracks where in the lesson the student is, what question they are on, and what the AI last said, so the backend can interpret follow-up voice inputs correctly (e.g. 'repeat that' or 'I said B' requires knowing what was just asked).

State	Description
IDLE	No active learning session. Student has not started a note. Voice triggers are: 'open [subject]', 'continue [subject]', 'what notes do I have'.
LISTENING_LESSON	Student is listening to a learning unit being read aloud by TTS. Voice triggers are: 'pause', 'repeat', 'next', 'go back', 'slower', 'stop', 'summarise this'.
QUESTION_POSED	AI has posed a question to the student via TTS and is awaiting voice answer. Student can say the answer, ask for the question to be repeated, or skip.
AWAITING_ANSWER	Backend is processing the student's voice answer. Client plays a brief 'thinking' audio tone. Backend scores the answer and prepares feedback.
FEEDBACK_PLAYING	TTS is reading feedback (correct/incorrect + explanation). Student can say 'next question', 'repeat', or 'explain more'.
FREE_ASK_MODE	Student has invoked the AI assistant ('ask something'). Student can ask any question about the current lesson content. Backend answers strictly from lesson text (RAG). Student exits with 'go back to lesson'.
SESSION_COMPLETE	Student has completed all units and questions in a note. Summary statistics are read aloud. Session is synced to backend.

The current session state is maintained on the Android device (in-memory + Room DB). The backend is stateless per-request but reconstructs context from the session_id and the current_unit_id / current_artefact_id passed with every voice API request. The backend MUST validate that the incoming request is consistent with a valid state transition.

21.3 New Database Entities for Voice Interaction

21.3.1 Entity: voice_sessions

Tracks the lifecycle of each student's learning session at a note level.

Column	Type & Notes
id	UUID PK
student_id	UUID FK -> users.id NOT NULL
note_id	UUID FK -> lesson_notes.id NOT NULL
started_at	TIMESTAMPTZ NOT NULL — when student opened the note
ended_at	TIMESTAMPTZ — NULL if session is still in progress or was abandoned
end_reason	ENUM('completed','abandoned','timed_out') — NULL if still active
units_completed	INTEGER DEFAULT 0
questions_answered	INTEGER DEFAULT 0
questions_correct	INTEGER DEFAULT 0

total_voice_inputs	INTEGER DEFAULT 0 — count of voice utterances in this session
free_ask_count	INTEGER DEFAULT 0 — how many times student used Free Ask mode
device_id_hash	VARCHAR(64)
synced_at	TIMESTAMPTZ
offline_started_at	TIMESTAMPTZ NOT NULL — actual device time

21.3.2 Entity: voice_interactions

Detailed log of every voice exchange — the raw transcript, interpreted intent, action taken, and response delivered. This is the core audit trail for research and model improvement.

Column	Type & Notes
id	UUID PK
session_id	UUID FK -> voice_sessions.id NOT NULL
student_id	UUID FK -> users.id NOT NULL
unit_id	UUID FK -> learning_units.id — context unit at time of interaction
artefact_id	UUID FK -> ai_artefacts.id — if interaction was tied to a specific question
sequence_number	INTEGER NOT NULL — order within session
raw_audio_duration_ms	INTEGER — length of the student's voice input in ms
transcription_source	ENUM('on_device','backend_whisper') NOT NULL
transcript_text	TEXT NOT NULL — what the student said (transcribed)
asr_confidence	FLOAT — 0.0–1.0 confidence from ASR engine
intent	VARCHAR(80) NOT NULL — classified intent (see Section 21.4)
intent_confidence	FLOAT — 0.0–1.0 NLU classification confidence
entities	JSONB — extracted entities: {answer_choice, unit_ref, subject, etc.}
action_taken	VARCHAR(80) — what backend did: scored_answer, fetched_next_unit, etc.
response_text	TEXT NOT NULL — text that was sent back for TTS playback
response_type	ENUM('feedback','navigation','answer','error','clarification') NOT NULL
processing_latency_ms	INTEGER — time from request receipt to response (backend processing only)
offline_recorded_at	TIMESTAMPTZ NOT NULL
synced_at	TIMESTAMPTZ

Index: CREATE INDEX idx_voice_interactions_session ON voice_interactions(session_id, sequence_number);

21.3.3 Entity: free_ask_exchanges

Dedicated table for student free-form questions posed to the AI assistant during Free Ask Mode. Stores the question, the matched lesson context, and the answer — critical for research analysis of what students are confused about.

Column	Type & Notes
id	UUID PK
interaction_id	UUID FK -> voice_interactions.id NOT NULL
student_id	UUID FK -> users.id NOT NULL
note_id	UUID FK -> lesson_notes.id NOT NULL
question_text	TEXT NOT NULL — the student's transcribed question
matched_unit_ids	UUID[] — array of learning_unit IDs whose text was used to answer
answer_text	TEXT NOT NULL — the answer generated from lesson text
similarity_scores	JSONB — {unit_id: cosine_similarity_score} map for each matched unit
answer_was_found	BOOLEAN NOT NULL — FALSE if no relevant content found in lesson
fallback_response	TEXT — response used when answer_was_found=FALSE
created_at	TIMESTAMPTZ DEFAULT now()

21.4 Intent Taxonomy — Complete Voice Command Set

The NLU classifier MUST recognise and correctly route all of the following intents. Intents are grouped by the session state they are valid in. The backend must reject (with a helpful clarification response) intents that arrive in invalid states.

21.4.1 Navigation Intents (valid in LISTENING_LESSON state)

Intent ID	Example Utterances	Action	TTS Response Pattern	State Transition
NAV_PAUSE	'pause', 'stop', 'wait'	Signal client to pause TTS playback	'Paused. Say continue when ready.'	LISTENING_LESSON -> IDLE (paused)
NAV_CONTINUE	'continue', 'go on', 'resume'	Signal client to resume TTS	'Continuing...'	IDLE (paused) -> LISTENING_LESSON
NAV_REPEAT	'repeat', 'say again', 'what did you say'	Return current unit text from start	Re-reads current learning unit from beginning	No state change

NAV_NEXT_UNIT	'next', 'next part', 'move on', 'I understand'	Fetch next learning unit	Reads unit heading aloud then begins unit text	LISTENING_LESSON (next unit)
NAV_PREV_UNIT	'go back', 'previous', 'back one'	Fetch previous learning unit	Re-reads previous unit from beginning	LISTENING_LESSON (prev unit)
NAV_SLOWER	'slower', 'speak slowly', 'too fast'	Return speech_rate adjustment hint	'Slowing down.' + speech_rate: 0.75 in response	No state change
NAV_FASTER	'faster', 'speed up'	Return speech_rate adjustment hint	'Speeding up.' + speech_rate: 1.25 in response	No state change
NAV_JUMP_TO	'go to [heading]', 'jump to [topic]'	Fuzzy match heading in current note; return unit_id	'Going to [matched heading].'	LISTENING_LESSON (jump)
NAV_STOP_SESSION	'stop lesson', 'exit', 'close'	Finalise voice_session record; trigger sync	'Session saved. Your progress has been recorded.'	Any -> SESSION_COMPLETE

21.4.2 Answer Intents (valid in QUESTION_POSED state)

Intent ID	Example Utterances	Action	TTS Response Pattern	State Transition
ANS_MCQ	'A', 'B', 'C', 'D', 'option A', 'I think it is B', 'the answer is C'	Extract answer_choice entity; score against correct_option_id; return feedback	Correct or incorrect + explanation text	QUESTION_POSED -> AWAITING_ANSWER -> FEEDBACK_PLAYING
ANS_SHORT	Any phrase not matching a single letter; natural language answer	Send transcript to short-answer scorer (cosine similarity); return score + model answer	Score feedback + model answer read aloud	QUESTION_POSED -> AWAITING_ANSWER -> FEEDBACK_PLAYING
ANS_REPEAT_Q	'repeat the question', 'say the question again', 'what was the question'	Return current artefact question_text	Re-reads question text	No state change
ANS_SKIP	'skip', 'next question', 'I do not know', 'pass'	Log as skipped (is_correct=NULL, skipped=TRUE); advance to next artefact	'Okay, moving on. The answer was: [correct answer].'	QUESTION_POSED -> FEEDBACK_PLAYING

ANS_HINT	'hint', 'give me a clue', 'help'	Return explanation field of artefact (which is derived from lesson text)	Reads the explanation as a hint without revealing full answer	No state change; hint_count incremented

21.4.3 AI Assistant Intents (Free Ask Mode — any state)

Intent ID	Example Utterances	Action	TTS Response Pattern	State Transition
ASK_INVOKE	'ask something', 'I have a question', 'can I ask'	Enter Free Ask Mode; inform student	'Sure, ask your question about this lesson.'	Any -> FREE_ASK_MODE
ASK_QUESTION	Any question utterance while in FREE_ASK_MODE	Run RAG retrieval against current note's learning units; generate grounded answer	Answer text read aloud (max 3 sentences)	FREE_ASK_MODE (stays)
ASK_ELABORATE	'tell me more', 'explain further', 'I still do not understand'	Fetch adjacent learning units; return expanded explanation from lesson text	Extended explanation from lesson content	FREE_ASK_MODE (stays)
ASK_EXIT	'go back', 'back to lesson', 'continue lesson', 'done asking'	Exit Free Ask Mode; return to prior state	'Back to the lesson.'	FREE_ASK_MODE -> prior state
ASK_SUMMARY	'summarise', 'give me a summary', 'what is this about'	Return summary artefact for current unit	Reads summary_text of current learning unit	No state change
ASK_KEY_TERMS	'what are the key terms', 'vocabulary', 'important words'	Return key_terms from summary artefact	Reads each key term with a pause between each	No state change

21.4.4 Session Management Intents (any state)

Intent ID	Example Utterances	Action	TTS Response Pattern	State Transition
SESS_PROGRESS	'how am I doing', 'my score', 'progress'	Fetch student's progress for current note from session state	'You have completed [N] of [M] sections. Your score so far is [X]%.'	No state change
SESS_WHERE_AM_I	'where am I', 'which part', 'current topic'	Return current unit heading and position	'You are on part [N] of [M]: [unit heading].'	No state change

SESS_OPEN_NOTE	'open [subject]', 'study [topic]', 'play [note title]'	Fuzzy match note title/subject; return first unit of matched note	Reads note title and first unit heading, then begins unit	IDLE -> LISTENING_LESSON
SESS_LIST_NOTES	'what notes do I have', 'my lessons', 'show subjects'	Return list of assigned notes for student	Reads each note title and subject aloud with numbering	IDLE (stays)
SESS_REPEAT_FEEDBACK	'repeat feedback', 'say that again', 'what did I get'	Return last response_text from voice_interactions for session	Re-reads last feedback or response	No state change
UNKNOWN	Anything not matched with confidence >= 0.60	Return clarification prompt	'I did not understand. You can say: next, repeat, answer A, ask something, or stop.'	No state change

21.5 New Voice API Endpoints

> **BASE PATH:** All voice endpoints are under /api/v1/voice. All require JWT authentication (student role minimum). All support gzip compression.

21.5.1 POST /voice/transcribe

Used only as fallback when on-device ASR confidence is below 0.65, or when the student is offline and has queued audio for transcription during next sync. Accepts raw audio and returns transcription.

Attribute	Specification
Method	POST
Path	/api/v1/voice/transcribe
Auth	JWT — student or teacher
Content-Type	multipart/form-data
Request Body	audio_file: binary (WAV or OGG, 16kHz mono, 16-bit, max 15s); language: string (default 'en', supports 'lg' for Luganda); session_id: UUID
Processing	Whisper small.en model (English); Whisper small multilingual for Luganda. Both run server-side. GPU inference if available; CPU fallback.
Response	{ transcript: string, confidence: float, language_detected: string, duration_ms: integer, processing_ms: integer }

Latency Target	< 1200ms for 15s audio on CPU (Whisper small.en: ~0.5x real-time on modern CPU)
Error Cases	Audio too short (< 300ms) → 400 AUDIO_TOO_SHORT. Audio too long (> 15s) → 400 AUDIO_TOO_LONG. Unintelligible audio → 200 with transcript: '' and confidence: 0.0
Offline Handling	If student was offline when voice was captured, audio clips can be batched in a POST /voice/transcribe/batch call on next sync (up to 20 clips). Backend transcribes all and returns array of results.

21.5.2 POST /voice/interpret

Takes a transcript (from on-device ASR or from /voice/transcribe) and the current interaction context, and returns the classified intent, extracted entities, and the backend action + response text. This is the central NLU endpoint.

Attribute	Specification
Method	POST
Path	/api/v1/voice/interpret
Auth	JWT — student
Content-Type	application/json
Request Body	{ transcript: string, session_id: UUID, current_unit_id: UUID, current_artefact_id: UUID|null, session_state: string (ENUM from Section 21.2), note_id: UUID, language: string }
NLU Pipeline	1. Normalize transcript (lowercase, strip filler words: 'um', 'uh', 'like'). 2. Run zero-shot intent classifier (transformers pipeline, model: facebook/bart-large-mnli quantised to INT8). 3. Extract entities with spaCy NER + custom patterns for answer choices (A/B/C/D). 4. Validate intent against current session_state. 5. Dispatch to action handler.
Action Dispatch	Intent handler executes synchronously within the request: score answer (if ANS_*), fetch unit text (if NAV_*), run RAG (if ASK_QUESTION). Response text is generated and returned in same HTTP response.
Response	{ intent: string, intent_confidence: float, entities: object, action_taken: string, response_text: string, response_type: string, speech_rate: float (default 1.0, may be adjusted), next_state: string, payload: object|null (e.g. next_unit data for NAV_NEXT_UNIT) }
Latency Target	< 600ms for navigation/answer intents; < 1500ms for ASK_QUESTION (RAG lookup); < 200ms for UNKNOWN/clarification
Low-Confidence Handling	If intent_confidence < 0.60 → force intent = UNKNOWN regardless of top prediction; return clarification response_text

21.5.3 POST /voice/session/start

Attribute	Specification
Method	POST
Path	/api/v1/voice/session/start
Auth	JWT — student
Request Body	{ note_id: UUID, resume_unit_id: UUID|null (null = start from beginning or last saved position) }
Action	Create voice_session record. If resume_unit_id is null, determine resume point from student_progress for this note. Return first (or resume) unit data + first artefact for that unit.
Response	{ session_id: UUID, unit: LearningUnit, artefacts: AIArtefact[], unit_position: int, total_units: int, intro_text: string (TTS greeting) }
Intro text example	'Welcome back. You are on part 3 of 7: Chemical Reactions. You had answered 4 questions in this section. Continuing from where you left off.'

21.5.4 POST /voice/session/event

Lightweight endpoint for logging voice interaction events. Called after every voice exchange. Designed for minimal payload and maximum throughput.

Attribute	Specification
Method	POST
Path	/api/v1/voice/session/event
Auth	JWT — student
Request Body	{ session_id: UUID, sequence_number: integer, unit_id: UUID, artefact_id: UUID|null, transcript_text: string, asr_confidence: float, transcription_source: string, intent: string, intent_confidence: float, entities: object, action_taken: string, response_text: string, response_type: string, processing_latency_ms: integer, offline_recorded_at: ISO8601 }
Action	Insert voice_interactions record. Update voice_sessions.total_voice_inputs counter. If action_taken = 'scored_answer', also insert student_progress record.
Response	{ ok: true, interaction_id: UUID } — intentionally minimal for speed
Latency Target	< 100ms — this is called after every single voice utterance
Offline Queue	If student is offline, events are queued locally in Room DB. All queued events MUST be flushed via POST /voice/session/events/batch on next sync.

21.5.5 POST /voice/session/end

Attribute	Specification

Method	POST
Path	/api/v1/voice/session/end
Auth	JWT — student
Request Body	{ session_id: UUID, end_reason: string, units_completed: integer, questions_answered: integer, questions_correct: integer, total_voice_inputs: integer, free_ask_count: integer, ended_at: ISO8601 }
Action	Finalise voice_session record. Compute session statistics. Update student_progress for all completed units in this session. Trigger analytics aggregation task.
Response	{ session_summary: { score_percent: float, units_completed: int, time_spent_seconds: int, encouragement_text: string } }
Encouragement text	Server generates contextual TTS-ready encouragement based on score: >= 80% → strong praise, 50–79% → positive with motivation, < 50% → supportive with suggestion to revisit. Text is read aloud by Android client after session.

21.5.6 POST /voice/ask (Free Ask Mode — RAG Endpoint)

This is the most complex voice endpoint. It powers Free Ask Mode: the student asks any natural language question about the lesson, and the backend answers strictly from the uploaded lesson text using Retrieval-Augmented Generation.

Attribute	Specification
Method	POST
Path	/api/v1/voice/ask
Auth	JWT — student
Request Body	{ question_text: string, note_id: UUID, current_unit_id: UUID, session_id: UUID, language: string }
RAG Pipeline	See Section 21.6 for full RAG specification
Response	{ answer_text: string, answer_found: boolean, source_unit_ids: UUID[], confidence: float, fallback_used: boolean }
Latency Target	< 1500ms (retrieval + generation, CPU-bound)
No-answer case	If no sufficiently relevant content found (max similarity < 0.45): answer_found=false; fallback response: 'I could not find an answer to that in this lesson. The lesson covers [key_terms from summary]. Try asking about one of those topics.'
Safety constraint	Backend MUST verify that answer_text contains ONLY content derivable from the note's learning units. A post-generation grounding check verifies no sentence in the answer has cosine similarity < 0.40 to any unit in the note. Failing sentences are removed before returning.

21.5.7 GET /voice/session/{session_id}/progress

Attribute	Specification
Method	GET
Path	/api/v1/voice/session/{session_id}/progress
Auth	JWT — student (own sessions only)
Action	Returns real-time progress within current session: units done, current unit heading, score so far, questions remaining. Used by SESS_PROGRESS intent.
Response	{ units_completed: int, total_units: int, current_unit_heading: string, score_percent: float, questions_answered: int, questions_remaining: int, time_elapsed_seconds: int }
Response text	Backend also returns a pre-formatted tts_summary string ready for immediate TTS playback, e.g. 'You have completed 3 of 7 sections. Your score is 75 percent. You have 4 questions remaining in this section.'

21.5.8 POST /voice/session/events/batch (Offline Sync)

Used during sync window to flush all voice interaction events that were recorded offline.

Attribute	Specification
Method	POST
Path	/api/v1/voice/session/events/batch
Auth	JWT — student
Request Body	{ events: Array<VoiceInteractionEvent>, sessions: Array<VoiceSessionRecord> } — same schema as individual endpoints but batched
Max batch	500 events per request
Action	Deduplication by (session_id, sequence_number). Insert all non-duplicate records. Update student_progress for all scored answers in batch. Update voice_sessions records.
Response	{ processed_events: int, skipped_duplicates: int, session_count: int, errors: [] }
Latency Target	< 2000ms for 500-event batch

21.6 Free Ask Mode — RAG Pipeline Specification

This pipeline is triggered by the ASK_QUESTION intent. It retrieves the most relevant portions of the uploaded lesson content and generates a grounded, spoken-language answer. It is strictly content-bound — the system cannot use any knowledge outside the uploaded note.

Step 1: Query Embedding

- Normalise student question: lowercase, remove filler words, correct common ASR errors (via a small phonetic correction dictionary).
- Embed the normalised question using sentence-transformers (paraphrase-MiniLM-L6-v2, INT8 quantised). This model runs entirely server-side.
- Question embedding is a 384-dimension vector.

Step 2: Corpus Retrieval

- All learning units for the note are pre-embedded at index time (during generate_artefacts task) and stored as JSONB embedding vectors in the learning_units table.
- Compute cosine similarity between question embedding and all unit embeddings for the note.
- Retrieve top-3 units by cosine similarity score. Minimum similarity threshold: 0.40. If no unit meets threshold → answer_found=false.
- If top unit score > 0.75: use only top-1 unit (highly targeted answer). If top score 0.40–0.75: use top-3 units (broader context).

Step 3: Context Window Assembly

- Concatenate retrieved unit texts in similarity-rank order, separated by a blank line.
- Prepend an instruction prefix: 'Answer the following question using ONLY the text below. If the answer is not in the text, say so. Question: [question_text]. Text: [retrieved units].'
- Total context window MUST NOT exceed 1024 tokens (enforced by truncation of lower-ranked units).

Step 4: Answer Generation

- Pass assembled context to a lightweight extractive-abstractive generator: first attempt pure extraction (select the most answer-relevant sentence from retrieved units using a BERT extractive QA model — distilbert-base-cased-distilled-squad, INT8 quantised).
- If extractive span confidence >= 0.70: use extracted span as the answer base. Expand to include the surrounding 1–2 sentences for context.
- If extractive confidence < 0.70: fall back to template: 'Based on the lesson, [most relevant sentence from top unit].'
- Answer MUST be converted to conversational spoken form: no markdown, no bullet points, written in complete sentences suitable for TTS. Max 3 sentences.

Step 5: Grounding Validation

- Split generated answer into sentences. For each sentence, compute cosine similarity to all retrieved unit sentences.
- If any answer sentence has maximum similarity < 0.40 to all unit sentences → remove that sentence from the answer (hallucination guard).
- If grounding check removes all sentences → return no-answer fallback.

Step 6: Response Packaging

- Return answer_text (spoken-form, grounded), source_unit_ids (list of units used), confidence (top similarity score), answer_found: true.
- Store exchange in free_ask_exchanges table for research analytics.

21.7 NLU Model Specification

The NLU pipeline runs entirely on the backend server. No external AI API calls are made. All models are downloaded at deployment time and stored on the server.

Model / Component	Specification
Intent Classifier	facebook/bart-large-mnli (zero-shot classification via NLI). Candidate labels = full intent taxonomy (Section 21.4). INT8 quantised for CPU efficiency. Model size: ~500MB quantised. Inference: ~150ms per classification on 4-core CPU.
Entity Extractor	spaCy en_core_web_sm + custom pattern matcher. Custom patterns cover: answer choices (A/B/C/D and ordinals), unit references ('part 3', 'section two'), subject names (from school curriculum list), and common navigation phrases. No neural overhead for entity extraction.
Sentence Embeddings	sentence-transformers/paraphrase-MiniLM-L6-v2. 384-dim embeddings. INT8 quantised. Model size: ~25MB. Inference: ~10ms per sentence on CPU. Used for: RAG retrieval, short-answer scoring, grounding validation.
Extractive QA	distilbert-base-cased-distilled-squad. INT8 quantised. Model size: ~65MB. Inference: ~100ms on CPU. Used for: Step 4 of Free Ask RAG pipeline.
ASR (Transcription)	openai/whisper-small.en for English. openai/whisper-small (multilingual) for Luganda detection. Both run on CPU via whisper.cpp (C++ binding, ~5x faster than Python). Model size: 244MB each. Inference: ~0.5x real-time (15s audio transcribed in ~7s on 4-core CPU).
Filler Word List	Static list of ASR noise terms to strip before NLU: 'um', 'uh', 'like', 'you know', 'err', 'hmm', 'so', 'well'. Applied as pre-processing step.
ASR Error Dictionary	Phonetic correction map for common ASR errors on African-accented English (e.g. 'be' → 'B', 'see' → 'C', 'dee' → 'D', 'hay' → 'A' — critical for MCQ answer detection).
Model Storage	All models stored at /opt/models/ on server. Loaded into memory at worker startup. NOT reloaded per-request.
Model Update Policy	Models are frozen at deployment. Updates require explicit versioning and regression testing — not automatic. Version tracked in ai_artefacts.generation_model field.

21.8 Answer Scoring via Voice Input

When a student answers a question by voice, the scoring pipeline must handle both MCQ and short-answer types from a natural language voice transcript.

21.8.1 MCQ Voice Answer Scoring

1. Receive transcript_text from /voice/interpret request.
2. Normalise: lowercase, strip filler words, apply ASR error corrections.
3. Extract answer entity: look for patterns — single letter ('a','b','c','d'), ordinal ('first','second','third','fourth'), or phrase match to option text.
4. Entity extraction confidence: if pattern matched → confidence 1.0. If fuzzy match to option text (cosine similarity >= 0.70) → confidence = similarity score.
5. If no answer entity extracted with confidence >= 0.60: return UNKNOWN intent → ask student to repeat ('I did not catch your answer. Say A, B, C, or D.').
6. Compare extracted answer_choice to correct_option_id. Compute: is_correct (boolean), score (1.0 or 0.0 for MCQ).
7. Generate feedback response_text: if correct → 'Correct! [explanation text from artefact]'. If incorrect → 'Not quite. The correct answer is [option text]. [explanation text].'
8. Store in student_progress via voice_interactions record.

21.8.2 Short Answer Voice Scoring

9. Receive full transcript_text as the student's answer.
10. Embed transcript using paraphrase-MiniLM-L6-v2.
11. Embed model_answer (the source sentence stored in artefact).
12. Cosine similarity score → 0.75+ = correct (1.0), 0.50–0.74 = partial (0.5), < 0.50 = incorrect (0.0).
13. Generate voice feedback: correct → 'Well done! That is correct.' + encouragement. Partial → 'You are on the right track. A more complete answer would be: [model answer].' Incorrect → 'Not quite. The answer is: [model answer]. Let us read that part again.' (trigger re-read of source unit).
14. Log score, student_answer (transcript), and is_correct in student_progress.

21.9 Offline Voice Interaction Fallback Specification

The backend cannot be reached when the student is offline. The Android client must continue to provide a fully functional audio learning experience. The backend's responsibility here is to pre-deliver everything the client needs to operate voice interaction without a connection.

> **DESIGN RULE:** The sync-pull payload (GET /api/v1/sync/pull) MUST include all data required for offline voice interaction. The Android client must never need to call a backend endpoint during a learning session. Backend voice endpoints are only called when online.

Additional Data in Sync-Pull for Voice Interaction

Data Element	Purpose for Offline Voice

Learning units (all text_content)	Android TTS reads these aloud to student
AI artefacts (question_text, options, summary, key_terms)	Android client poses questions during session; NO correct answers included
intent_model_bundle	Lightweight on-device intent classifier: a compressed (.tflite or ONNX) intent classification model (~10MB) pre-compiled for the 30-odd intent classes. Downloaded once; used offline for all sessions. Backend pushes updated bundle version when model is retrained.
asr_correction_map	JSON phonetic correction dictionary for common ASR errors. Downloaded with sync. Used by Android SpeechRecognizer post-processing.
filler_words_list	JSON list of filler words to strip. Downloaded with sync.
feedback_templates	Per-artefact feedback_template strings for offline feedback delivery. The template for each question is included in the artefact payload.
tts_intro_text	Per-note welcome/intro text ready for TTS (avoids client having to construct it).
navigation_phrases_map	Maps common navigation phrases to intent IDs for lightweight offline matching (keyword fallback when on-device model confidence is low).

Offline Intent Matching Flow

15. Android SpeechRecognizer produces transcript (offline ASR, always available).
16. Run on-device intent model (TFLite). If confidence >= 0.65 → dispatch intent locally.
17. If confidence < 0.65 → fall back to navigation_phrases_map keyword lookup.
18. If still unmatched → play clarification prompt from static audio file: 'I did not understand. You can say: next, repeat, answer A, or stop.'
19. All interactions are logged in Room DB. Voice interactions and session events are queued for sync.
20. Answer scoring (MCQ only) runs fully offline: correct_option_id is revealed after completion via /sync/answers; until then, Android stores answer_choice and defers scoring to next sync.

21.10 Performance Requirements — Voice Endpoints

Endpoint	Latency Target (P99)
POST /voice/transcribe (15s audio, CPU)	< 1200ms
POST /voice/transcribe (5s audio, CPU)	< 500ms
POST /voice/interpret — navigation intent	< 300ms

POST /voice/interpret — answer scoring (MCQ)	< 400ms
POST /voice/interpret — answer scoring (short answer)	< 600ms
POST /voice/interpret — UNKNOWN / clarification	< 200ms
POST /voice/ask (RAG — 3 unit retrieval + generation)	< 1500ms
POST /voice/ask (no-answer fallback)	< 300ms
POST /voice/session/event	< 100ms
POST /voice/session/start	< 400ms
POST /voice/session/end	< 400ms
POST /voice/session/events/batch (500 events)	< 2000ms

Voice Performance Optimisation Rules

- All NLP models MUST be loaded into memory at Celery worker startup — NOT per-request. Cold load penalty is paid once per worker lifecycle.
- POST /voice/interpret MUST execute synchronously within the HTTP request — no task queue for navigation or answer intents. Latency must be felt in real-time conversation.
- POST /voice/ask MAY be dispatched to a high-priority Celery task if server load is high — but MUST complete and return within 1500ms. Use async HTTP long-poll or WebSocket upgrade if needed.
- Voice endpoints MUST have their own Uvicorn worker pool separate from general API workers — isolate model memory from REST endpoints.
- Intent model inference MUST run on a dedicated thread pool (ThreadPoolExecutor) to avoid blocking the async event loop.
- Whisper transcription MUST run in a subprocess or thread — it is CPU-bound and must not block the async event loop.

21.11 Amendments to Existing Data Models

The following columns are added to existing tables defined in the original PRD (Section 5) to support voice interaction:

Additions to: learning_units

New Column	Type & Purpose

embedding_vector	JSONB — 384-dim float array (sentence-transformers embedding). Computed during generate_artefacts task. Used for RAG cosine similarity retrieval. Nullable until processed.
tts_intro_text	TEXT — pre-formatted spoken introduction: 'Part [N] of [M]: [heading]. [first sentence of unit].' Generated server-side for consistency.

Additions to: ai_artefacts

New Column	Type & Purpose
feedback_correct_text	TEXT — TTS-ready feedback for correct answer. e.g. 'Correct! [explanation].' Stored for offline use.
feedback_incorrect_text	TEXT — TTS-ready feedback for incorrect answer. e.g. 'Not quite. The answer is [correct option text]. [explanation].' Stored for offline use.
feedback_partial_text	TEXT — TTS-ready feedback for partial score. e.g. 'You are on the right track. A complete answer would be: [model answer].'
hint_text	TEXT — TTS-ready hint derived from explanation. 'Here is a clue: [explanation without revealing answer].'
skip_reveal_text	TEXT — TTS-ready text read when student skips. 'The answer to that question was: [correct option text]. [explanation].'

Additions to: student_progress

New Column	Type & Purpose
voice_session_id	UUID FK -> voice_sessions.id — links progress event to voice session
interaction_id	UUID FK -> voice_interactions.id — links to specific voice exchange
hint_used	BOOLEAN DEFAULT FALSE — whether student requested a hint before answering
answer_attempt_type	ENUM('voice_mcq','voice_short_answer','offline_mcq') — how the answer was submitted
skipped	BOOLEAN DEFAULT FALSE — whether student skipped this question

21.12 New Background Jobs for Voice System

Task Name	Specification
precompute_unit_embeddings	Triggered by: generate_artefacts task completion. For each learning unit, compute sentence-transformer embedding and store in learning_units.embedding_vector. Priority: High. Required BEFORE

	the note is marked 'ready' — must block note status update until all unit embeddings are computed. Estimated duration: 500ms per unit.
generate_voice_artefact_texts	Triggered by: precompute_unit_embeddings completion. Generate all TTS-ready feedback texts for every ai_artefact in the note (feedback_correct_text, feedback_incorrect_text, feedback_partial_text, hint_text, skip_reveal_text). These are template-based — fast. Estimated duration: 100ms per artefact.
compile_intent_model_bundle	Triggered by: manual system_admin action or model update event. Compiles the on-device TFLite intent model bundle for Android client download. Output stored at /opt/models/intent_bundle_v{N}.tflite. Adds bundle version to sync-pull metadata. Runs as a one-off task.
aggregate_voice_analytics	Scheduled: daily 02:30 UTC (30 min after main analytics job). Aggregates: avg_free_ask_per_session, avg_intent_confidence, top_confused_questions, sessions_completed, sessions_abandoned, avg_session_duration_minutes. Stores in analytics_events table.
cleanup_voice_sessions	Scheduled: every 6 hours. Mark voice_sessions as 'timed_out' if started_at < now() - 24 hours and ended_at is NULL. These were abandoned without proper session end.

21.13 Voice-Specific Security Requirements

- Audio files received by POST /voice/transcribe MUST be validated for actual audio format using python-magic before processing. Reject non-audio files with 400 INVALID_AUDIO.
- Audio files MUST be stored in a temporary directory (/tmp/voice/) with a random UUID filename. The file MUST be deleted immediately after transcription. Audio is never persisted.
- Transcripts ARE stored in voice_interactions.transcript_text. Developers must be aware this is personal data (the student's own spoken words). It MUST be excluded from any data exports not explicitly consented to by the user.
- The voice/ask RAG endpoint MUST enforce a rate limit of 20 ASK_QUESTION requests per student per hour to prevent abuse of the NLP pipeline.
- POST /voice/interpret and POST /voice/ask MUST validate that the provided note_id is actually assigned to the authenticated student. A student MUST NOT be able to query lesson content from notes not assigned to them.
- Intent model bundles distributed to Android clients MUST be signed with a server private key. The Android client MUST verify the signature before loading a new bundle.

21.14 Testing Requirements — Voice System

Unit Tests

- Test intent classifier with: 20 correctly-classified utterances per intent (all intents in Section 21.4). Target: >= 90% accuracy on held-out test set.
- Test MCQ answer entity extraction with: 'A', 'option A', 'I think A', 'the answer is A', 'first option', 'see' (phonetic for C), 'dee' (phonetic for D).

- Test short-answer scorer with: exact match, strong paraphrase (expect correct), weak paraphrase (expect partial), completely wrong answer (expect incorrect).
- Test RAG pipeline with: factual question answerable from unit, factual question NOT answerable from unit, question that requires combining two units, nonsense question.
- Test grounding validation: inject a synthetic hallucinated sentence; verify it is removed by grounding check.

Integration Tests

- Full session simulation: POST /voice/session/start → 3x POST /voice/interpret (navigation) → 2x POST /voice/interpret (answer MCQ) → 1x POST /voice/ask → POST /voice/session/end. Verify all DB records created correctly.
- Offline batch sync: create 100 voice events in local format; POST to /voice/session/events/batch; verify all inserted with correct deduplication.
- State validation: submit ANS_MCQ intent while in LISTENING_LESSON state; verify backend returns clarification error (not an answer score).

Performance Tests

- POST /voice/transcribe: test with 10 concurrent 15-second audio clips; all must complete in < 1200ms P99.
- POST /voice/ask: test with 20 concurrent RAG requests against a 10-unit note; all must complete in < 1500ms P99.
- Model startup time: verify all NLP models loaded and warm within 30 seconds of worker start.

21.15 Acceptance Criteria — Voice System

The voice interaction system is considered complete when ALL of the following pass:

21. A student can start a learning session, navigate through all units, and complete all questions using ONLY voice commands — zero screen touches required.
22. MCQ answer intents correctly extract the answer choice from 'A', 'option B', 'I think it is C', and phonetic variants 'see', 'dee', 'ay', 'bee' with >= 90% accuracy on a 50-utterance test set.
23. Short answer scoring returns correct / partial / incorrect feedback with appropriate TTS response text for each.
24. The UNKNOWN intent is returned for any utterance with intent_confidence < 0.60, and the clarification response_text lists valid commands.
25. Free Ask Mode answers a factual question from lesson text within 1500ms and the answer contains no content not present in the uploaded note (grounding check passes).
26. Free Ask Mode correctly returns answer_found=false for a question with no relevant content in the lesson.
27. POST /voice/session/event responds in < 100ms P99 under 50 concurrent requests.
28. The sync-pull payload includes the intent_model_bundle, asr_correction_map, and all TTS feedback texts for offline use.

29. A full simulated offline session (50 events) is successfully batch-synced via /voice/session/events/batch with zero data loss.
30. Audio files are confirmed deleted from /tmp/voice/ within 5 seconds of transcription completion.
31. A student cannot call /voice/ask on a note not assigned to them — returns 403.

21.16 Updated Development Phases — Voice System Integration

Phase (Voice Work)	Sprint & Deliverables
Phase 3B: Voice Foundation	Sprint 5 (alongside AI Pipeline Phase 3): Whisper setup + /voice/transcribe endpoint; spaCy entity patterns for answer choices; POST /voice/session/start and /end endpoints; voice_sessions and voice_interactions tables.
Phase 4B: NLU + Interaction Loop	Sprint 7 (alongside Sync Phase 4): BART-MNLI intent classifier integration; full intent taxonomy routing; /voice/interpret endpoint with all navigation and answer intents; MCQ and short-answer scoring via voice; TTS feedback text generation in artefact pipeline.
Phase 4C: RAG + Free Ask	Sprint 8: sentence-transformers unit embedding pipeline; /voice/ask RAG endpoint; grounding validation; free_ask_exchanges table; Free Ask Mode intents.
Phase 4D: Offline Voice Bundle	Sprint 8: TFLite intent model compilation; intent_model_bundle in sync-pull; asr_correction_map; feedback_templates in sync payload; offline batch sync endpoint.
Phase 5B: Voice Analytics	Sprint 9 (alongside Analytics Phase 5): voice session aggregation; confused questions analysis; intent confidence analytics; voice_sessions cleanup job.
Phase 7B: Voice Hardening	Sprint 11 (alongside Hardening Phase 7): voice endpoint load testing; audio file deletion audit; intent accuracy regression tests; RAG grounding audit with 20 sample notes.

21.17 Open Questions — Voice System

#	Question	Impact
V1	Should Luganda voice interaction be supported from launch? Whisper multilingual supports Luganda ASR, but the intent classifier and RAG pipeline are English-only. A Luganda NLU layer would require a separate intent model and Luganda training data.	Significant scope increase; may be Phase 2
V2	What is the wake word / invocation mechanism? Options: (a) tap screen + speak, (b) always-listening wake word ('Hey VisioLearn'), (c)	UX specification for Android client

	volume-button press. Backend is not affected, but the choice affects the UX design of the voice session start flow.	
V3	Should the backend store raw audio permanently for research (with student consent)? This enables future model training but introduces significant data privacy obligations and storage cost.	Privacy, storage, consent framework
V4	For the pilot, will voice sessions need a teacher monitoring view? Real-time or async? This determines whether a WebSocket teacher dashboard is needed in Phase 6.	Admin dashboard scope
V5	Should the system support a 'repeat lesson' voice command that re-reads the entire unit at a student-chosen speed? This is trivial for the client but the backend must return speed_preference storage in user profile.	User profile schema

— END OF AMENDMENT A —

VisioLearn Backend PRD — Amendment A: Voice Interaction System | v1.1 | March 2026

This document should be read alongside and supersedes relevant sections of VisioLearn Backend PRD v1.0